

Escuela Politécnica Superior, Ingeniería
Informática

Nombre y apellidos:

1. Marcelo Fort Muñoz

2. Victor Arroyo Márquez

Prácticas de
Sistemas Fiables
en Tiempo Real
25/26



UNIVERSIDAD
NEBRIJA

Nombre y apellidos

Índice/Tabla de contenidos

Índice/Tabla de contenidos.....	2
1. INTRODUCCIÓN.....	4
2. EJERCICIOS.....	4
SESIÓN 1.....	4
Los ejercicios de la primera sesión fueron introductorios. Se ha pedido que no se incluyan, por lo que nos abstendremos de comentarlos en esta memoria.....	4
SESIÓN 2.....	4
EJERCICIO 2.1.....	4
EJERCICIO 3.1.....	5
EJERCICIO 4.1.....	6
EJERCICIO 5.1.....	8
EJERCICIO 6.1.....	10
.....	11
SESIÓN 3.....	11
EJERCICIO 2.1.....	11
EJERCICIO 3.1.....	17
EJERCICIO 4.1.....	21
EJERCICIO 5.1.....	22
EJERCICIO 5.1.....	23
EJERCICIO 6.1.....	23
.....	25
RESUMEN y CONCLUSIONES.....	25
A. Anexo I: Códigos de las prácticas.....	26
SESIÓN 2.....	26
EJERCICIO 2.1.....	26
EJERCICIO 3.1.....	29
EJERCICIO 4.1.....	32
EJERCICIO 5.1.....	37
SESIÓN 3.....	40
EJERCICIO 2.1.....	40
B. Anexo II: Licencias del contenido.....	45

1. INTRODUCCIÓN

En esta memoria se presentarán los resultados y conclusiones a las prácticas realizadas en los laboratorios de la asignatura Sistemas Tolerantes a Fallos en Tiempo Real. Se ha programado en el microcontrolador ESP32S3 diferentes programas y se ha hecho una batería de pruebas para ver como reacciona este a diferentes formas de planificación y carga.

2. EJERCICIOS

SESIÓN 1

Los ejercicios de la primera sesión fueron introductorios. Se ha pedido que no se incluyan, por lo que nos abstendremos de comentarlos en esta memoria.

SESIÓN 2

EJERCICIO 2.1

En el ejercicio 2.1 se pide crear dos tareas a distintas frecuencias (3 Hz en GPIO N.º 4 y 6 Hz en el GPIO N.º 5) en el ecosistema FREERTOS. Después se pide implementar un sistema de control que permita suspender la tarea 2 (controladora del GPIO N.º 5) al pulsar el botón integrado de la ESP32. Después de volver a pulsar el botón y, tras un retardo de 3000 ms, la tarea se reanudará. El estado de la tarea 2 es impreso continuamente por la terminal del sistema para poder monitorear su funcionamiento y la interacción con el botón «BOOT».

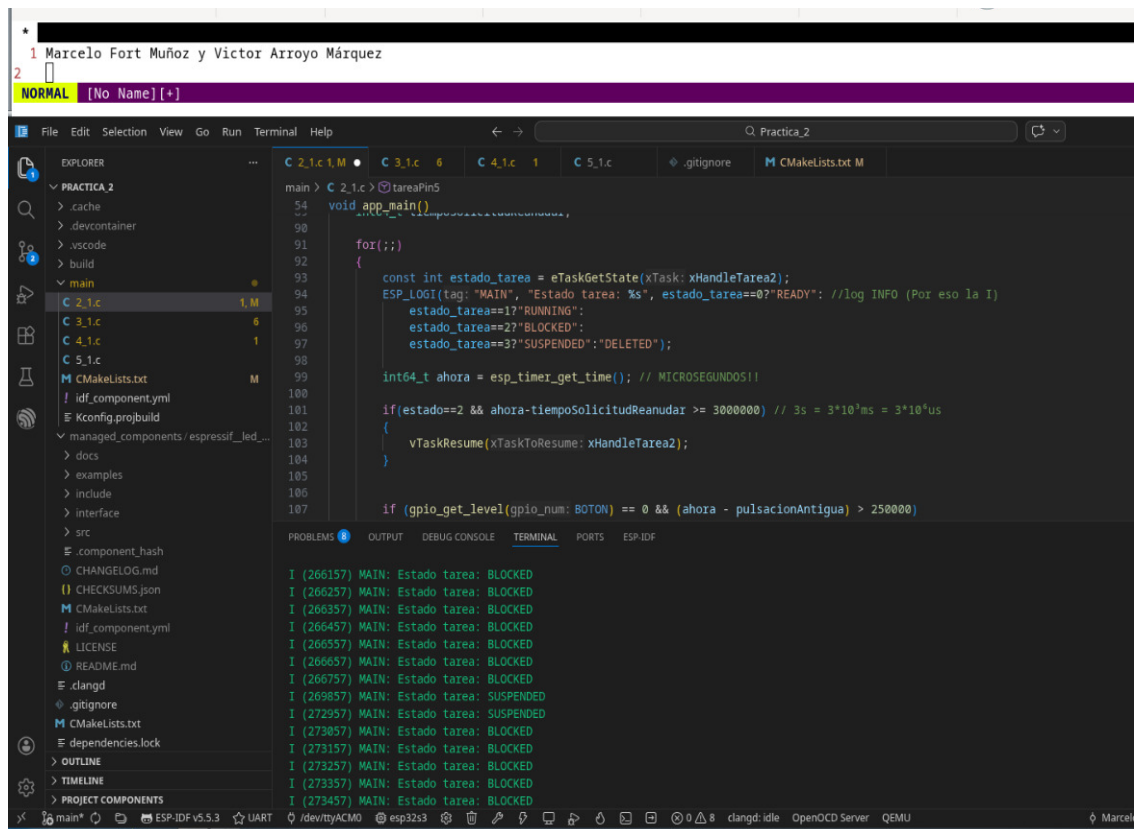


Figura 1: Ejercicio 2.1 de la Práctica 2

Podemos observar en la captura mostrada en la Figura 1 que hasta el instante 266.757 ms, la tarea estaba en estado bloqueada, es decir, disponible para ser planificada. En ese instante, podemos entender que el botón integrado ha sido presionado, por lo que pasa de bloqueada a suspendida, ya no siendo planificable.

Después se vuelve a pulsar el botón y, en el instante 273.057 ms, presumiblemente tras los 3.000 ms de espera vuelve a entrar a la planificación de tareas, siendo dada CPU en algún momento futura por el planificador. Decidimos como tamaño de cola 2 KiB, dando un poco de margen libre como indican las buenas prácticas de programación de tiempo real.

EJERCICIO 3.1

En este ejercicio, se pide que se creen dos tareas: Una leerá un ADC ubicado en el ADC N.º 5 (ubicado en el GPIO 6) y otra realizará la media móvil con los datos leídos de este sensor integrado en el chip. La tarea del ADC realiza la lectura cada 0.2 segundos. Posteriormente, la segunda tarea recibe el dato mediante una cola y realiza la media con magnitud 10. Después de hacer la media, la imprime en la consola.

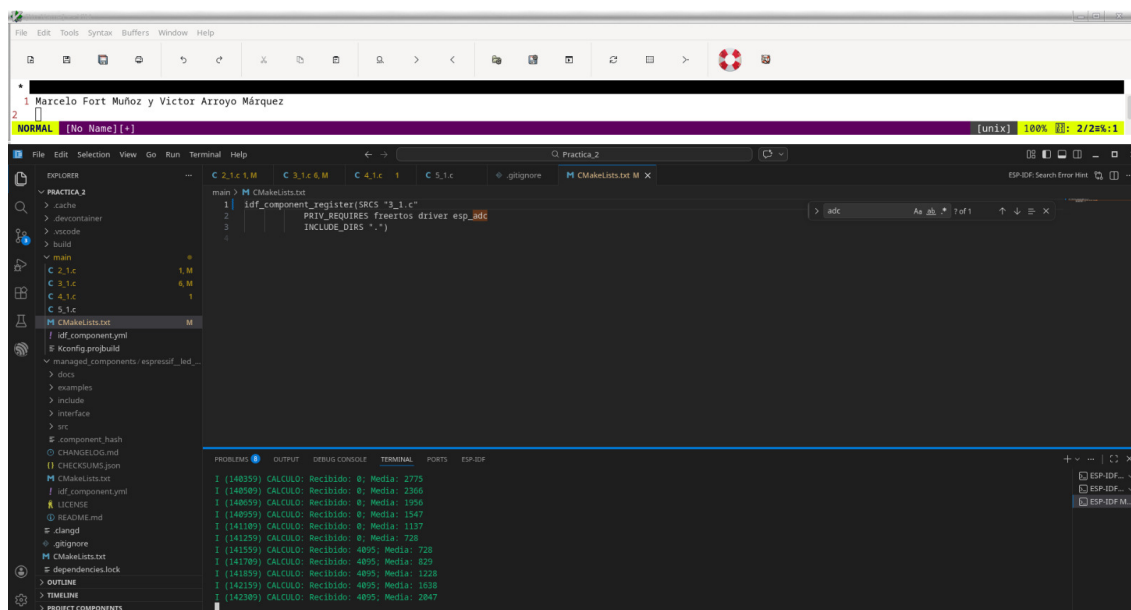


Figura 2: Ejercicio 3.1 de la práctica 2, ejemplo de cambio brusco en los valores de entrada.

Podemos observar en la Figura 2, la media móvil tiene una gran utilidad para suavizar respecto a grandes entradas o cambios bruscos en los valores de esta. En el ejemplo podemos ver como ante una elevación repentina a 4095 en vez de saltar de 728 a 4095, los valores van creciendo paulatinamente, permitiendo de esta forma reducir el error al que podrían inducir datos atípicos o lecturas incorrectas. Es un filtro eficaz para suavizar entradas variables para sistemas que requieren cierta consistencia. Se eligió un tamaño mayor que la ventana de la media para poder observar pérdidas de datos cuando la tarea productora genera más rápido que la consumidora, tal como indica el enunciado. Este sistema se usa en electrónica para filtrar sensores, en finanzas para análisis de mercados y tendencias, para eliminar ruido en audios y en telemetría para gestionar datos de sensores en tiempo real. Tiene muchos otros usos en distintos ámbitos, pero esta es una buena lista para hacerse una idea

EJERCICIO 4.1

En este ejercicio, se nos pidió, en primer lugar, habilitar el time slicing, es decir, «trocear» en secciones de tiempo (quantum) el tiempo de CPU y distribuir el quantum entre las diferentes tareas. El uso de quantum (NICE en Windows), permite que cuando los procesos tienen igual prioridad, se use el sistema round robin. Volviendo al ejercicio, se piden tres tareas «concurrentes» que compartan un ADC ubicado en el sexto pin protegido por mutex.

Todas las tareas miden el mutex 1.000 cada vez que pueden. La tarea 1 mide el menor valor. La tarea 2 hace un promedio y, la tarea tres busca la lectura más alta.

Decidimos 2 KiB de pila para las tareas para que «printf» pudiese tener memoria suficiente para imprimir en la terminal. El ADC es compartido por las tareas y se les pasa cómo parámetro. Además, las tareas reservan la memoria de las lecturas del montón, no necesitando de esta forma memoria de pila para sus operaciones.

Podemos observar claramente como funciona el programa en la Figura 3.

```
main > CMakeLists.txt
1 | idf_component_register(SRCS "4_1.c"
2 |   PRIV_REQUIRES freertos driver esp_adc
3 |   INCLUDE_DIRS ".")
4 |

Tarea 3 lectura promedio: 0 | tiempo: 50004
Tarea 1 coge mutex: 50006
Tarea 1 lectura minima: 0 | tiempo: 50071
Tarea 2 coge mutex: 50072
Tarea 2 lectura máxima: 72 | tiempo: 50137
Tarea 3 coge mutex: 50138
Tarea 3 lectura promedio: 0 | tiempo: 50204
Tarea 1 coge mutex: 50206
Tarea 1 lectura minima: 0 | tiempo: 50271
Tarea 2 coge mutex: 50272
Tarea 2 lectura máxima: 72 | tiempo: 50337
Tarea 3 coge mutex: 50338
Tarea 3 lectura promedio: 0 | tiempo: 50404
Tarea 1 coge mutex: 50406
```

Figura 3: Ejecución del programa 4_1 de la práctica 2

EJERCICIO 5.1

En este ejercicio se implementa un sistema concurrente compuesto por tres tareas sincronizadas mediante un semáforo.

La Tarea 1 genera una señal periódica de 10 Hz en un LED conectado a GPIO4, funcionando como indicador de actividad del sistema.

La Tarea 2 actúa como interfaz de usuario mediante la detección de la pulsación del botón BOOT (en el flanco de bajada). Al detectarse el evento, imprime un mensaje por la terminal, alterna el estado del LED en GPIO5 y libera la ejecución de la Tarea 3 mediante un semáforo.

La Tarea 3 permanece bloqueada hasta recibir dicha señal, garantizando una ejecución estrictamente basada en eventos. Una vez activada, realiza 10 mediciones del ADC en GPIO6, calcula el valor medio y lo muestra por la terminal.

En la Figura 4 se observa el comportamiento del sistema, donde cada pulsación del botón desencadena una única ejecución de la Tarea 3, validando la correcta sincronización entre tareas y el modelo implementado.

```
1 Marcelo Fort Muñoz y Victor Arroyo Márquez
2
```

```
main > C 5_1.c > tarea1
22 void tarea1(void *params)
27
28     for (;;)
29     {
30         gpio_set_level(gpio_num: LED1, level: estado_led);
31         estado_led = !estado_led;
```

PROBLEMS 9 OUTPUT DEBUG CONSOLE TERMINAL PORTS ESP-IDF

```
¡Botón pulsado!
Promedio ADC: 1706
¡Botón pulsado!
Promedio ADC: 1709
¡Botón pulsado!
Promedio ADC: 1709
¡Botón pulsado!
Promedio ADC: 1707
¡Botón pulsado!
Promedio ADC: 1711
¡Botón pulsado!
Promedio ADC: 1708
¡Botón pulsado!
Promedio ADC: 1712
```

main* New ESP-IDF Hints! ESP-IDF v5.5.3 UART /dev/ttyACM0 esp32s3

Figura 4: Ejercicio 5.1 de la práctica 2

EJERCICIO 6.1

Se pide coger un ejercicio y pasar la función de estadísticas. Escogimos el ejercicio 5. Según podemos ver en Figura 5, la tarea IDLE consume el que más tiempo visto con los porcentajes. El resto de las tareas consumen menos del 1% de uso. La tarea 2 es la que más tiempo de CPU consume, seguida de las tareas 3 y 1. El IDLE consume más del 99% del tiempo de CPU.

Posteriormente, probamos a añadir un «nop» en bucle a la tarea dos. Como era de esperar y podemos ver en Figura 6 esto causó que la tarea dos aumentase su tiempo de CPU, llegando a sobre el 11% y, disminuyendo el tiempo de IDLE al 88%

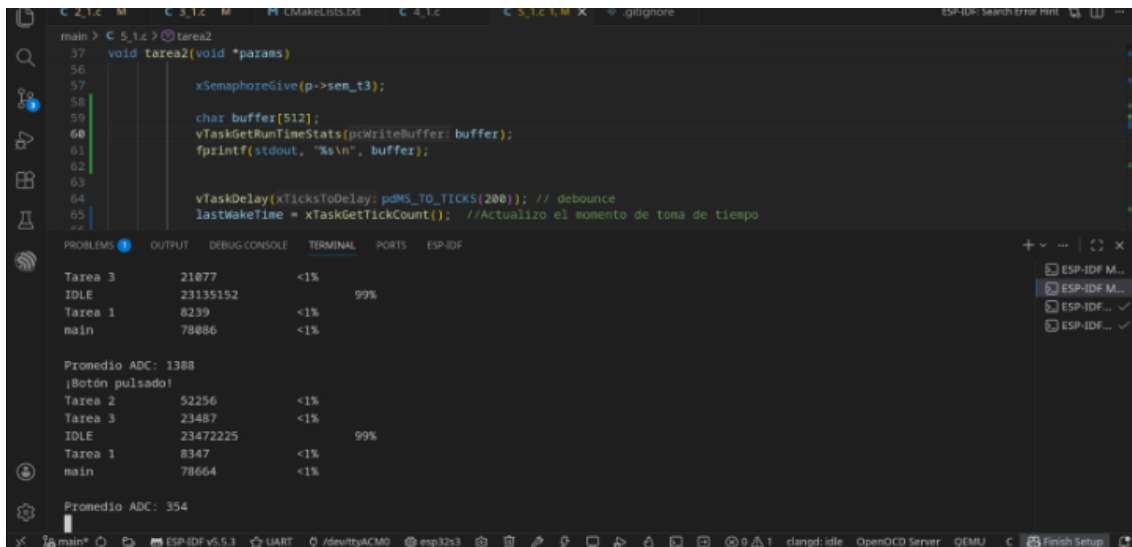
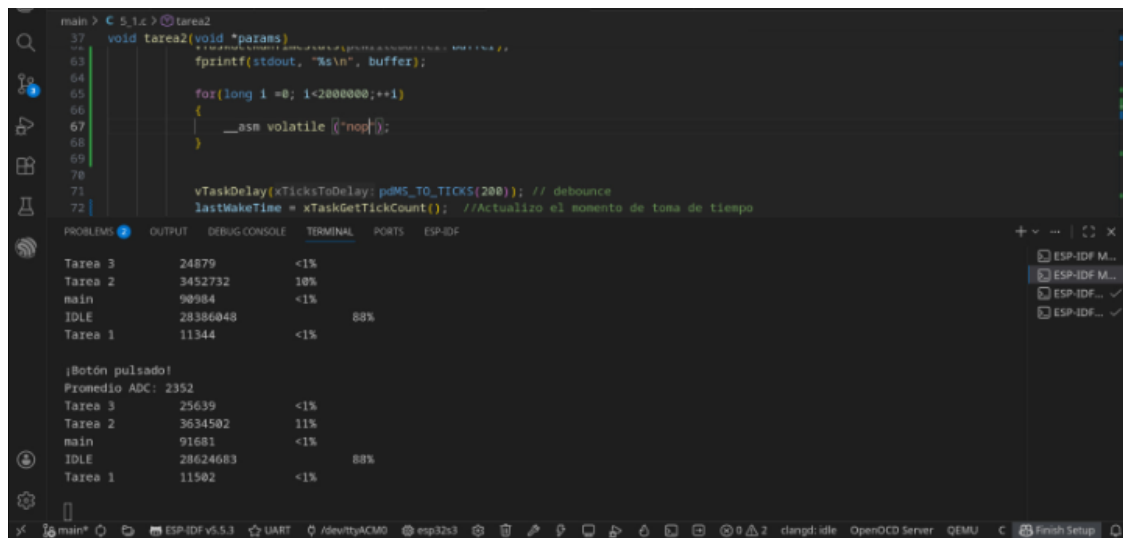


Figura 5: Ejercicio 6.1 sobre 5.1 sin NOPs de la práctica 2.



Por lo tanto, podemos calcular la duración de cada tarea al saber que en su bucle principal cada una ejecutará 15000 NOPs:

$$DuracionTarea = 15000 \text{ ciclos} * 12.5 \text{ ns} = 187.500 \text{ ns} = 187.5 \mu s = 0.1875 \text{ ms}$$

Dentro del periodo de 400 ms que tiene cada tarea, nos da tiempo de sobra, puesto que nos sobran:

$$U = (5 * 0.1875 \text{ ms}) / 400 \text{ ms} = 0.00234 = 0.234 \%$$

Lo que es tiempo de sobra para cumplir los plazos, la utilización es muy inferior al 100%. Todo esto sin tener en cuenta cambios de contexto e interrupciones. Pero el margen es tan grande que se puede asegurar que la planificación tendrá lugar con éxito y holgura.

Todo esto lo hemos llevado a la práctica y como podemos ver en la Figura 7, la ejecución da un tiempo holgado, como prometíamos antes. Un lector avisado podría decaerse de que el tiempo ocioso de la CPU no es de 99.8% como asegurábamos en el párrafo anterior. Esto se debe a que el planificador funciona con un tick de 1 KHz. Esto produce interrupciones y cambios de contexto regulares, incluso cuando las tareas sí mantienen el comportamiento predicho (uso menor al 1%).

Esta sobrecarga del sistema operativo introduce aproximadamente un 4 % de consumo de CPU asociado a gestión del planificador, lo que reduce el tiempo ocioso observado hasta alrededor del 96 %.

Como en nuestra explicación teórica, los márgenes se cumplen y podemos ver que la infrautilización penaliza en un RTOS, al «pagar» los cambios de contexto sin aprovecharlos.

Respecto al resto de preguntas (respondidas con datos obtenidos empíricamente):

La tarea tres tarda en ejecutarse entre dos y tres segundos, teniendo una latencia de esa magnitud, que será igual para todas las tareas, tanto la uno como la dos, cuatro o cinco. De esta forma, la latencia de una supuesta ejecución de todas las tareas sería (obviando overheads e interrupciones):

$$L = 5 * 2 \text{ ms} = 10 \text{ ms}$$

Visto todo esto, podemos afirmar que el planificador tiene suficientes recursos para acomodar una tarea N.º 6.

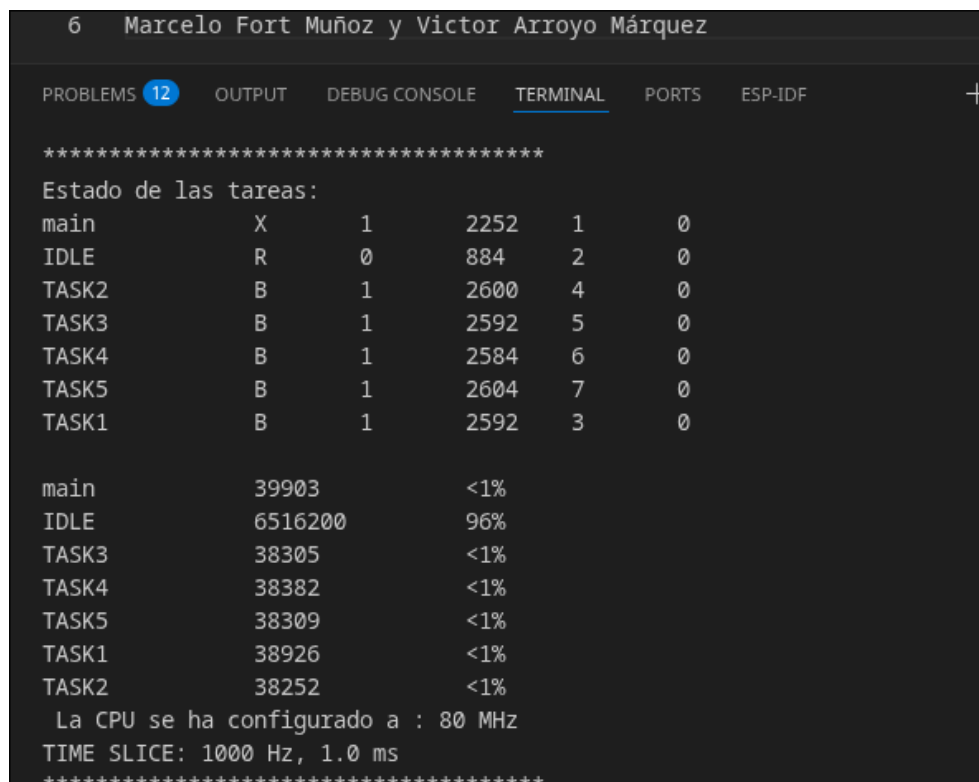


Figura 7: Ejecución del ejercicio 2.1 del laboratorio 3.

EJERCICIO 2.2

Hemos ajustado los periodos a 200 ms y el número de instrucciones NOP a 100.000. Tras hacer esto, volvemos a ejecutar el programa. En los resultados observados en las figuras Figura 8 y Figura 9, podemos observar que, la tarea 4 dura $108-47=61$ ms, que oscilará en torno a 60 ms. Las tareas (todas) se ejecutan cada 200ms. En un caso específico de la 1 podemos observar: $244-38=206$ ms, que es una pequeña desviación de 200ms.

Se observa una ventana de ejecución de aproximadamente 70 ms ($308 - 240 = 68$ ms), correspondiente a un fragmento del planificador dentro de un ciclo, no a un ciclo completo del sistema. Todas las tareas tienen ejecuciones que duran (aproximadamente) lo mismo.

Mirando la Figura 9, observamos que, todas las tareas usan una cantidad muy similar de CPU, siendo el porcentaje común el 7%.

Podemos calcular el porcentaje de uso como $1 - \text{Idle} = 1 - 0.62 = 0.38 = 38\%$.

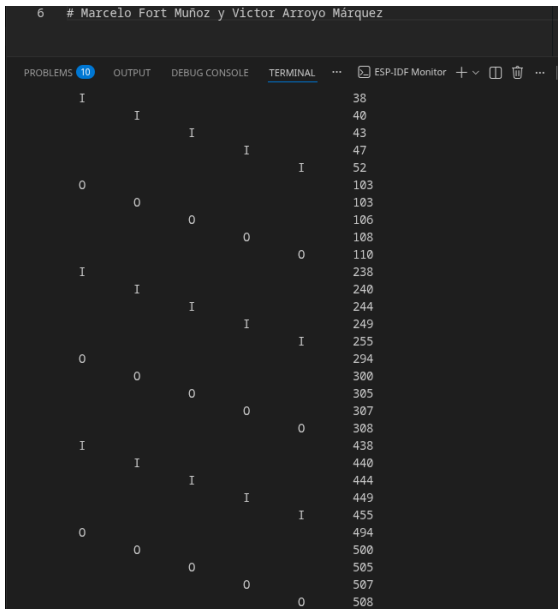


Figura 8: Ejecución del programa 2.2 de la práctica 3

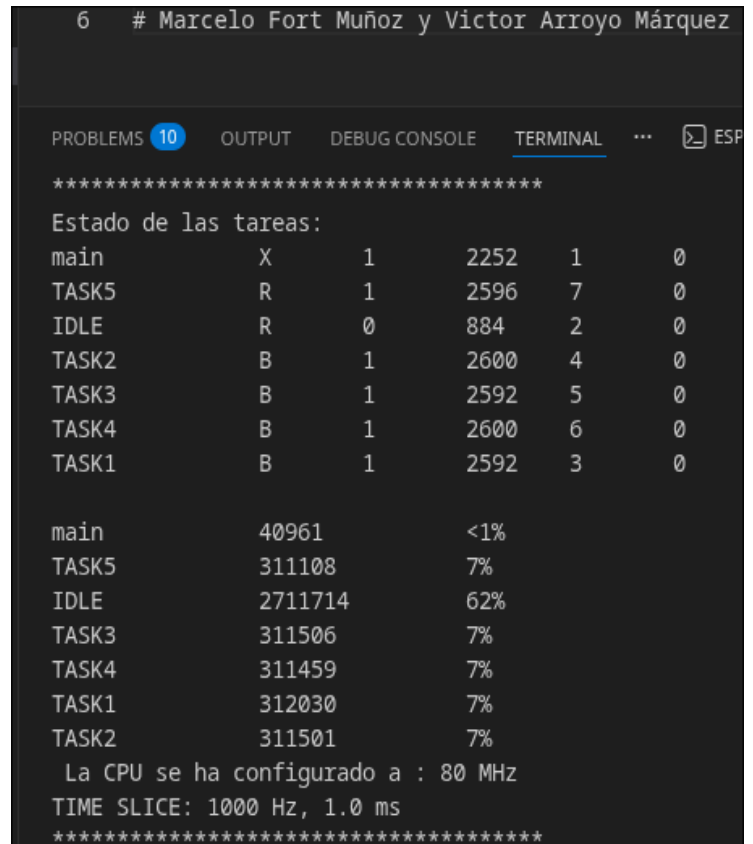


Figura 9: Estadísticas de la ejecución del programa 2.2 de la práctica 3

EJERCICIO 2.3

Hemos ajustado los periodos a 200 ms y el número de instrucciones NOP a 100.000. Tras esto, volvemos a ejecutar el programa. En los resultados observados en la Figura 12, se puede observar que las tareas presentan un comportamiento periódico estable, ejecutándose aproximadamente cada 200 ms.

La tarea 2 presenta una duración de ejecución en torno a 60 ms. En distintos intervalos se observa, por ejemplo, un comportamiento comprendido entre 1903–1954 ms (51 ms) y 1940–2004 ms (64 ms), por lo que su duración oscila aproximadamente entre 55 ms y 65 ms, soliendo ser de sobre 60 ms.

Se observa además que la tarea 2 presenta un comportamiento no completamente continuo, sino fragmentado en varias secciones de ejecución. Esto se debe al planificador

con time slicing, que interrumpe y reanuda la ejecución de la tarea, generando múltiples eventos O sin un único bloque continuo I-O.

La periodicidad de la tarea 2 es de aproximadamente 200 ms, como se puede comprobar observando las activaciones sucesivas (por ejemplo, 1903 ms y 1990 ms, así como en ciclos anteriores del sistema).

Se observa una ventana de ejecución de aproximadamente 70 ms ($308 - 240 = 68$ ms), correspondiente a un fragmento del planificador dentro de un ciclo, no a un ciclo completo del sistema.

En la Figura 12, correspondiente al uso de CPU se observa que todas las tareas presentan un uso muy similar, en torno al 7% cada una. El uso total de CPU puede calcularse como:

$$1 - \text{Idle} = 1 - 0.62 = 0.38 = 38\%$$

Podemos ver los valores empíricos en la

En la Figura 10 se puede ver el diagrama que representa el comportamiento esperado del sistema bajo planificación periódica. Todas las tareas se ejecutan de forma regular cada 200 ms, con duraciones de ejecución aproximadamente constantes (~60 ms), lo que refleja un reparto estable del tiempo de CPU bajo un esquema de planificación preemptiva con time slicing.

La Figura 11 representa el comportamiento observado durante la ejecución real del sistema. En este caso se aprecia que la tarea 2 no se ejecuta de forma continua, sino que su ejecución aparece fragmentada en múltiples ráfagas debido a la planificación preemptiva y a la asignación no uniforme de quantum de CPU entre tareas concurrentes.

La diferencia entre ambos diagramas no indica un fallo del sistema, sino el efecto del planificador bajo condiciones de carga y con tareas de igual prioridad.

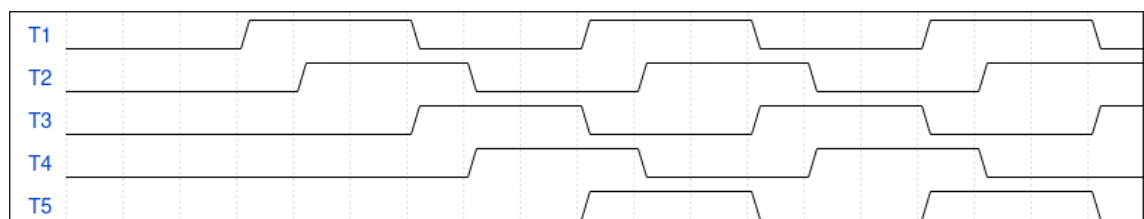


Figura 10: Diagrama de la ejecución del programa 2.3 de la práctica 3.



Figura 11: Diagrama de la ejecución del programa 2.3 de la práctica 3: Comportamiento de la tarea con un periodo incompatible.

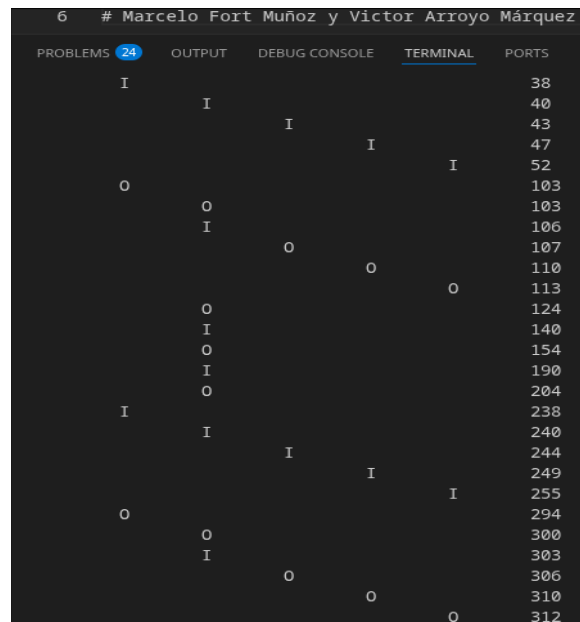


Figura 12: Comportamiento del ejercicio 2.3 de la práctica 3

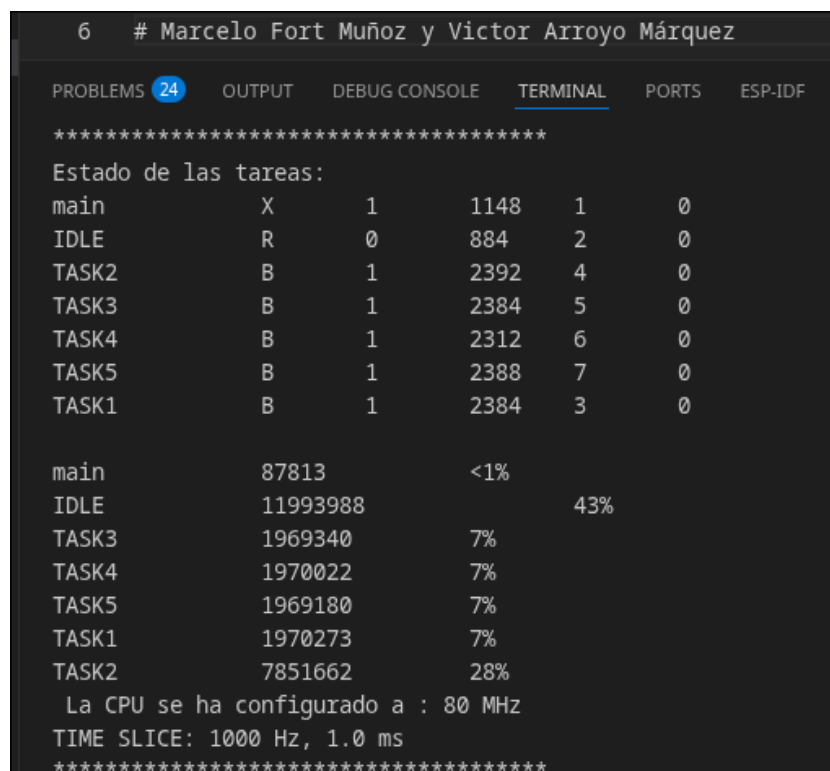


Figura 13: 2.3 de la práctica 3: Estadísticas

EJERCICIO 3.1

En este caso, a duración de cada tarea se mantiene aproximadamente constante porque todas ejecutan la misma carga de NOPs. La diferencia respecto al caso con *time slicing* no es el tiempo total de ejecución de cada tarea, sino cómo se distribuye en el tiempo: cada tarea aparece ejecutándose en bloques más largos y menos fragmentados, sin cortes periódicos como antes.

La frecuencia de ejecución de cada tarea viene dada por sus periodos configurados, por lo que cada tarea se repite aproximadamente cada 200 ms, pero como no hay *time slicing*, dentro de su ventana de ejecución ocupa la CPU de forma más continua. Esto hace que visualmente en el trazo se vean ejecuciones más “limpias” y menos interrumpidas.

El tiempo total en ejecutar el conjunto de tareas no cambia significativamente respecto al caso con *time slicing*, ya que depende de la carga global del sistema y no del reparto interno del planificador. Lo que sí cambia es la latencia y la forma de reparto de CPU entre tareas, que se incrementa.

En cuanto al uso de CPU, a partir de las estadísticas del sistema se observa que el tiempo de IDLE ronda aproximadamente el 30%, por lo que el uso total de CPU es aproximadamente $1 - 0.30 = 0.70$, es decir, un 70% de utilización. Esto encaja con la ausencia de *time slicing*, ya que hay menos overhead de cambios de contexto y menos fragmentación de ejecución, lo que mejora la eficiencia global del sistema.

Las tareas tardan en ejecutarse 140ms. Y cada tarea se ejecuta cada 200 ms.

Al reactivar el *time slicing*, las tareas siguen ejecutándose con el mismo periodo de 200 ms, pero su ejecución aparece fragmentada debido a las expropiaciones periódicas. El tiempo total necesario para completar todas las tareas dentro del periodo se mantiene en un orden similar (≈ 145 ms), observándose ligeras variaciones atribuibles al sobre-coste de los cambios de contexto.

El uso de la CPU en este caso es aproximadamente del 71%, una diferencia mínima respecto al caso sin *time slicing*, atribuible al margen de error de medida y no a una mejora real de eficiencia. El *time slicing* introduce un pequeño sobre-coste temporal debido a la gestión del planificador, aunque no altera de forma significativa la carga computacional total.

El aumento del tiempo de las tareas se debe a que el *time slicing* impone expropiaciones con cambio de contexto, que aumentan el sobre-coste temporal. Aunque el tiempo de CPU es casi el mismo para cada tarea, el sistema pierde tiempo saltando entre tareas. En sistemas muy predecibles y resistentes a bloqueos, una arquitectura cooperativa no preemptiva (sin *time slicing*) puede ser beneficiosa, al dejar que las tareas se coordinen entre ellas y eliminando esas expropiaciones arbitrarias.

A continuación en las figuras Figura 16 y Figura 17, ejemplificamos como se comporta según el tipo de fraccionado de tiempo que se use.

```

6 # Marcelo Fort Muñoz y Victor Arroyo Márquez

PROBLEMS 34 OUTPUT DEBUG CONSOLE TERMINAL PORTS I
I 8632
O 8659
I 8662
O 8689
I 8692
O 8719
I 8722
O 8749
I 8749
O 8777
I 8832
O 8859
I 8862
O 8889
I 8892
O 8919
I 8922
O 8949
I 8949
O 8977

*****
Estado de las tareas:
main X 1 2284 1 0
IDLE R 0 884 2 0
TASK2 B 1 2600 4 0
TASK3 B 1 2592 5 0
TASK4 B 1 2600 6 0
TASK5 B 1 2596 7 0
TASK1 B 1 2592 3 0

main 40300 <1%
IDLE 2711781 30%
TASK3 1245759 13%
TASK4 1245839 13%
TASK5 1246379 13%
TASK1 1246371 13%
TASK2 1245846 13%
La CPU se ha configurado a : 80 MHz
TIME SLICE: 100 Hz, 10.0 ms
*****

```

Figura 14: Ejercicio 3.1 p3: sin time slicing

```

      I                                     3632
          I                               3652
      0                                     3669
          I                               3682
          0                               3697
              I                           3712
              0                           3725
                  I                       3742
                  0                       3753
                      I                   3770
                      0                   3832
                          I               3852
                          0               3869
                              I           3882
                              0           3897
                                  I       3912
                                  0       3925
                                      I   3942
                                      0   3953

*****
Estado de las tareas:
main      X      1      2284      1      0
TASK5     R      1      2596      7      0
IDLE      R      0      884      2      0
TASK2     B      1      2600      4      0
TASK3     B      1      2592      5      0
TASK4     B      1      2600      6      0
TASK1     B      1      2592      3      0

TASK5     545354      13%
main      39367       <1%
IDLE      1160696     29%
TASK3     554141      13%
TASK4     554128      13%
TASK1     554394      13%
TASK2     554195      13%
La CPU se ha configurado a : 80 MHz
TIME SLICE: 100 Hz, 10.0 ms

```

Figura 15: Ejercicio 3.1 p3: con time slicing



Figura 16: Diagrama del 3.1 p3: sin time slicing

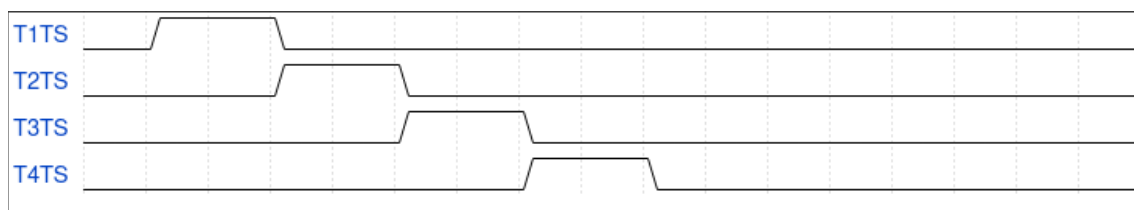


Figura 17: Diagrama del 3.1 p3: con time slicing

EJERCICIO 4.1

Las tareas tienen un tiempo de ejecución de sobre 70 ms. Se ejecutan cada 200 ms. Transcurre sobre un segundo para la ejecución de todas las tareas. La tarea 5 se ejecuta antes que la cuarta al tener más prioridad.

A partir de las estadísticas del sistema, el uso de CPU puede estimarse como $1 - 0.29 \approx 0.71$, es decir, aproximadamente un 71% de utilización. Este valor es coherente con la carga observada en el sistema y con el comportamiento del planificador con time slicing.

```

*****
Estado de las tareas:
main          X      1      2252      1      0
IDLE          R      0      884      2      0
TASK2         B      1      2600      4      0
TASK3         B      1      2592      5      0
TASK4         B      1      2600      6      0
TASK5         B      2      2284      7      0
TASK1         B      1      2592      3      0

main          42611      <1%
IDLE          7174838    29%
TASK3         3436567    14%
TASK4         3435386    14%
TASK5         3420960    14%
TASK1         3436456    14%
TASK2         3436200    14%

La CPU se ha configurado a : 80 MHz
TIME SLICE: 1000 Hz, 1.0 ms
*****

```

Figura 18: Captura ejecución 4-1 de 3

Respecto al ejercicio 4.2, la configuración de prioridades hace que la tarea de mayor prioridad se ejecute con preferencia sobre el resto cuando comparten estado listo, modificando la planificación respecto a un sistema sin esta diferenciación.

EJERCICIO 5.1

Como se puede ver en

No se cumple la periodicidad de las tareas de forma estricta, ya que aunque se activan cada 200 ms, la ejecución real se va retrasando debido a la sobrecarga del sistema.

Cada tarea se ejecuta aproximadamente cada 200 ms, pero los tiempos de inicio y final se van desplazando progresivamente.

El tiempo total necesario para ejecutar todas las tareas es de unos 350 ms (5 tareas de ~70 ms), lo que supera el periodo disponible de 200 ms. Esto implica que el sistema no tiene capacidad suficiente para completar todo dentro del ciclo.

La tarea 5 se ejecuta de forma diferente porque tiene mayor prioridad, lo que hace que preemta al resto de tareas y afecte al reparto normal de CPU.

El uso de CPU calculado es de 175% (350 ms / 200 ms), lo que indica claramente que el sistema está sobrecargado y no es viable en tiempo real.

Las estadísticas del sistema confirman una alta ocupación de CPU y falta de tiempo de ejecución para tareas de menor prioridad, incluido el IDLE.

En conclusión, el sistema está sobrecargado, no es planificable con estos tiempos y se producen retrasos acumulados que terminan provocando la activación del watchdog por falta de ejecución del IDLE.

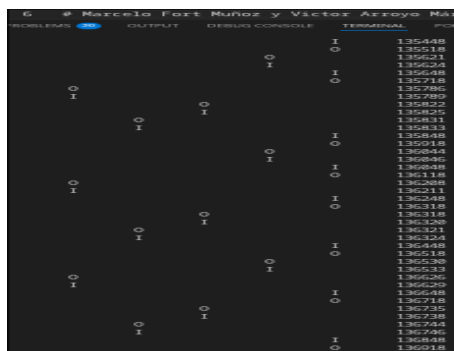


Figura 20: ejercicio 5.1 practica 3.



Figura 19: ejercicio 5.1 practica 3: estadísticas.

EJERCICIO 5.1

No se cumple la periodicidad de las tareas. Se hace un caso en la cual tarda 200 pero hay casos que tarda 418 que no debería tardar. 200. Sí, tiene más prioridad entonces, se ejecuta antes. El porcentaje de uso es de 1 -(>1%)=100%. No hace falta sumar nada al no cumplirse la periodicidad. El sistema está sobrecargado y no puede con las tareas

```
*****
Estado de las tareas:
TASK2      R      1      2600    4      0
TASK3      R      1      2592    5      0
TASK4      R      1      2600    6      0
main       X      1      2252    1      0
TASK1      R      1      2592    3      0
IDLE       R      0      1012    2      0
TASK5      B      2      2604    7      0

TASK2      362298      15%
TASK3      360815      15%
TASK4      359815      15%
main       40286       1%
TASK1      359838      15%
IDLE       0           <1%
TASK5      837962      36%

La CPU se ha configurado a : 80 Mhz
TIME SLICE: 1000 Hz, 1.0 ms
*****
```

Figura 21: captura practica 5.1 p3

EJERCICIO 6.1

La ISR dura aproximadamente 2.5 ms. Las tareas tienen un periodo fijo, es decir, se repiten cada 200 ms. Habiendo 5 tareas y todas con carga, $5 \times 2.5\text{ms} = 12.5\text{ms}$ (sin ISR), con ISR 2.5 ms extra. En el peor caso, $12.5\text{ms} + 2.5\text{ms} = 15\text{ms}$

la interrupción claramente afecta a las tareas. Por un lado las interrumpe para ejecutarse ella y, por el otro, ven desplazado su timestamp. En el log se puede apreciar el ruido.

```

6 # Marcelo Fort Muñoz y Víctor Arroyo Márquez
PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS ESP
      I      14082
      0      14097
      I      14112
      I      14122
      0      14149
      0      14163
      0      14170
      I      14232
      I      14252
      0      14269
      I      14282
      0      14297
      I      14312
      I      14322
      0      14349
      0      14363
      0      14370
      I      14432
      I      14452
      0      14469
      I      14482
      0      14497
      I      14512
      I      14522
      0      14549
      0      14563
      0      14570
      I      14632
      I      14652
      0      14669
      I      14682
      0      14697
      I      14712
*****14712
*****14740
      I      14740
      0      14768
      0      14771
      0      14798
      I      14832

```

*Figura 22: Captura log ejecución ej
6.1 practica 3*

EJERCICIO 7.1

No, al entrar en la sección crítica con ese candado se impide el robo de quantum por parte de otra tarea, Además, como la ISR tiene igual prioridad, la planificación se realiza en igualdad de oportunidades, o es decir, la coge el que llega primero. Podemos ver este comportamiento en la Figura 23.

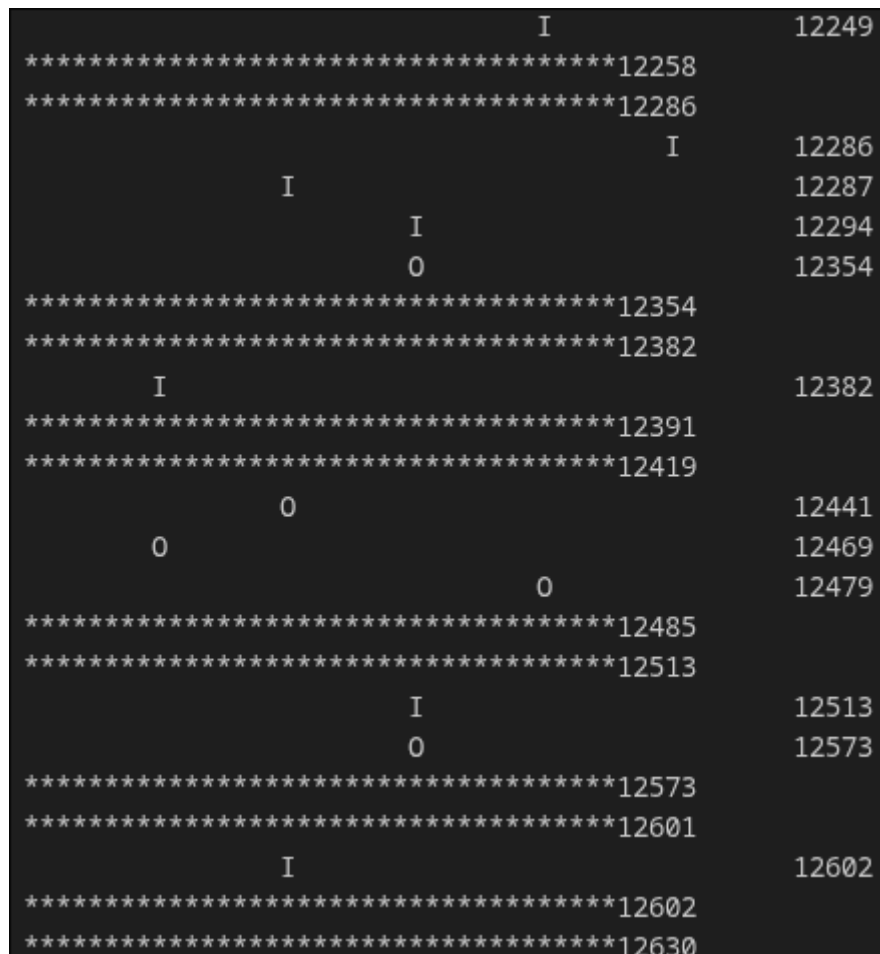


Figura 23: Comportamiento del ejercicio 7.1 de la práctica 3

RESUMEN y CONCLUSIONES

Durante estas sesiones de prácticas hemos descubierto varias cosas.

En primer lugar, que un correcto diseño del periodo y carga de las tareas es vital para la eficiencia y cumplimiento de plazos del sistema. También hemos podido observar como el time-slicing permite una distribución de quantum y como este difiere de una gestión cooperativa del tiempo de CPU, en la que las tareas tienen que ceder el uso. Por último, vimos como las interrupciones pueden forzar la expropiación a una tarea de alta prioridad y como evitarlo.

A. Anexo I: Códigos de las prácticas

SESIÓN 2

EJERCICIO 2.1

```
#include <stdio.h>
#include "esp_timer.h"
#include "freertos/FreeRTOS.h"
#include "freertos/projdefs.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "hal/gpio_types.h"
#include "portmacro.h"
#include "esp_log.h"

#define LED4 4
#define LED5 5
#define BOTON 0

#define APAGADO 0
#define ENCENDIDO 1

#define STACK_SIZE (2*1024)

void tareaPin4(void *pvParametros)
{
    const TickType_t xDelayTicks = pdMS_TO_TICKS(166);

    int estadoLed = APAGADO;

    TickType_t xLastWakeTime = xTaskGetTickCount();
    for(;;)
    {
        gpio_set_level(LED4, estadoLed);
        estadoLed=estadoLed==ENCENDIDO?APAGADO:ENCENDIDO;
        xTaskDelayUntil(&xLastWakeTime, xDelayTicks);
    }
}

void tareaPin5(void *pvParametros)
{
    const TickType_t xDelayTicks = pdMS_TO_TICKS(83);

    int estadoLed = APAGADO;

    TickType_t xLastWakeTime = xTaskGetTickCount();
    for(;;)
```

```

    {
        gpio_set_level(LED5, estadoLed);
        estadoLed=estadoLed==ENCENDIDO?APAGADO:ENCENDIDO;
        xTaskDelayUntil(&xLastWakeTime, xDelayTicks);
    }
}

void app_main()
{
    TickType_t xLastWakeTime = xTaskGetTickCount();

    gpio_set_direction(LED4, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED5, GPIO_MODE_OUTPUT);
    gpio_set_direction(BOTON, GPIO_MODE_INPUT);

    TaskHandle_t xHandleTarea1 = NULL;
    TaskHandle_t xHandleTarea2 = NULL;

    xTaskCreate(tareaPin4
        , "LED4"
        , STACK_SIZE
        , NULL
        , 1
        , &xHandleTarea1);

    xTaskCreate(tareaPin5
        , "LED5"
        , STACK_SIZE
        , NULL
        , 1
        , &xHandleTarea2);

    int64_t pulsacionAntigua = esp_timer_get_time();

    int estado = 0;
    int64_t tiempoSolicitudReanudar;

    for(;;)
    {
        const int estado_tarea = eTaskGetState(xHandleTarea2);
        ESP_LOGI("MAIN", "Estado tarea: %s", estado_tarea==0?"READY":
//log INFO (Por eso la I)
        estado_tarea==1?"RUNNING":
        estado_tarea==2?"BLOCKED":
        estado_tarea==3?"SUSPENDED":"DELETED");

        const int64_t ahora = esp_timer_get_time(); // MICROSEGUNDOS!!
    }
}

```

```

        if(estado==2 && ahora-tiempoSolicitudReanudar ≥ 3000000) //
3s = 3*103ms = 3*106us
        {
            vTaskResume(xHandleTarea2);
        }

        if (gpio_get_level(BOTON) == 0 && (ahora - pulsacionAntigua) >
250000)
        {
            pulsacionAntigua = ahora;

            estado =(estado+1)%3;

            switch(estado)
            {
                case 0:
                    break;
                case 1:
                    vTaskSuspend(xHandleTarea2);
                    break;
                case 2:
                    tiempoSolicitudReanudar = esp_timer_get_time();
                    break;
            }
            const TickType_t xDelayTicks = pdMS_TO_TICKS(3000);
            vTaskDelayUntil(&xLastWakeTime,
pdMS_TO_TICKS(xDelayTicks));
        }

        const TickType_t xDelayTicks = pdMS_TO_TICKS(100);
        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(xDelayTicks));
    }
}

```

EJERCICIO 3.1

```

/*
Copyright (C) 2026 Marcelo Fort Muñoz, Victor Arroyo Márquez

This program is free software: you can redistribute it and/or
modify it under the terms of the GNU General Public License as
published by the Free Software Foundation, either version 3 of
the License, or any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program. If not, see
<https://www.gnu.org/licenses/>.

*/

#include <stdio.h>
#include "esp_timer.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "esp_adc/adc_oneshot.h"

#define STACK_SIZE (4 * 1024) // 4 KiB
#define N 12

QueueHandle_t handleCola = NULL;

/* ----- */
void tareaLectura(void *pvParametros)
{
    const adc_oneshot_unit_handle_t adc1_handle =
        (adc_oneshot_unit_handle_t) pvParametros;

    const TickType_t xDelayTicks = pdMS_TO_TICKS(200);
    TickType_t xLastWakeTime = xTaskGetTickCount();

    for (;;)
    {
        int adc_raw = 0;
        adc_oneshot_read(adc1_handle, ADC_CHANNEL_5, &adc_raw);

        if (!xQueueSend(handleCola, &adc_raw, 0))
        {
            fprintf(stderr, "fallo en la cola (subir)\n");
        }
    }
}

```

```

xTaskDelayUntil(&xLastWakeTime, xDelayTicks);
}
}

/* ----- */
void tareaCalculo(void *pvParametros)
{
    const TickType_t xDelayTicks = pdMS_TO_TICKS(200);
    TickType_t xLastWakeTime = xTaskGetTickCount();

    int indice_elemento_lectura = 0;
    int datos_leidos[10] = {0};
    int muestras_validas = 0;

    for (;;)
    {
        if (xQueueReceive(handleCola,
            &datos_leidos[indice_elemento_lectura],
            portMAX_DELAY))
        {
            if (muestras_validas < 10)
            {
                muestras_validas++;
            }

            int media = 0;
            for (int i = 0; i < muestras_validas; i++)
            {
                media += datos_leidos[i];
            }

            media /= muestras_validas;

            ESP_LOGI("CALCULO",
                "Recibido: %i; Media: %i",
                datos_leidos[indice_elemento_lectura],
                media);

            indice_elemento_lectura =
                (indice_elemento_lectura + 1) % 10;
        }

        xTaskDelayUntil(&xLastWakeTime, xDelayTicks);
    }
}

/* ----- */
void app_main(void)
{
    handleCola = xQueueCreate(N, sizeof(int));

    adc_one_shot_unit_handle_t adc1_handle;

    adc_one_shot_unit_init_cfg_t init_config = {

```

```
        .unit_id = ADC_UNIT_1
    };
    adc_oneshot_new_unit(&init_config, &adc1_handle);

    adc_oneshot_chan_cfg_t config = {
        .atten = ADC_ATTEN_DB_12,
        .bitwidth = ADC_BITWIDTH_DEFAULT
    };
    adc_oneshot_config_channel(adc1_handle,
                               ADC_CHANNEL_5,
                               &config);

    xTaskCreate(tareaLectura,
                "LECTURA",
                STACK_SIZE,
                (void *) adc1_handle,
                1,
                NULL);

    xTaskCreate(tareaCalculo,
                "CALCULO",
                STACK_SIZE,
                NULL,
                1,
                NULL);

    for (;;)
    {
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

EJERCICIO 4.1

```

/*
Copyright (C) 2026 Marcelo Fort Muñoz, Victor Arroyo Márquez

This program is free software: you can redistribute it and/or
modify it under the terms of the GNU General Public License as
published by the Free Software Foundation, either version 3 of
the License, or any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program. If not, see
<https://www.gnu.org/licenses/>.

*/

#include <stdlib.h>
#include "freertos/FreeRTOS.h"
#include "freertos/projdefs.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp_adc/adc_oneshot.h"
#include "portmacro.h"

#define STACK_SIZE 2048

typedef struct
{
    SemaphoreHandle_t mutex;
    adc_oneshot_unit_handle_t adc;
} Parametros;

void tarea1(void* params)
{
    Parametros *parametros = (Parametros *) params;

    const TickType_t xDelayTicks = pdMS_TO_TICKS(200);
    TickType_t xLastWakeTime = xTaskGetTickCount();

    int lectura_minima=4096;
    for(;;)
    {
        if(!xSemaphoreTake(parametros->mutex, portMAX_DELAY))
        {
            continue;

```

```

}

printf("Tarea 1 coge mutex: %lu\n", (unsigned
long)xTaskGetTickCount()*portTICK_PERIOD_MS);
int *array_lecturas = (int*) pvPortMalloc(1000 * sizeof(int));
if (!array_lecturas)
{
    fprintf(stderr, "Tarea1: pvPortMalloc fallo\n");
    xSemaphoreGive(parametros->mutex);
    vTaskDelay(xDelayTicks);
    continue;
}

for (int i=0; i < 1000; ++i)
{
    adc_oneshot_read(parametros->adc, ADC_CHANNEL_5, array_lecturas+i);
}

    if(xSemaphoreGive(parametros->mutex)==pdFALSE)
    {
        fprintf(stderr, "Tarea1: error al soltar el mutex: %lu\n",
(unsigned long)xTaskGetTickCount()*portTICK_PERIOD_MS);
    }

    int lectura_mas_baja=array_lecturas[0];
    for(int i= 0; i<1000; ++i)
    {
        if(lectura_mas_baja >= *(array_lecturas+i))
        {
            lectura_mas_baja= array_lecturas[i];
        }
    }

    lectura_minima=lectura_mas_baja<lectura_minima?
lectura_mas_baja:lectura_minima;

    vPortFree(array_lecturas);
    printf("Tarea 1 lectura mínima: %d | tiempo:
%lu\n", lectura_minima, (unsigned
long)xTaskGetTickCount()*portTICK_PERIOD_MS);

    xTaskDelayUntil(&xLastWakeTime, xDelayTicks);
}
}

void tarea2(void* params)
{
    Parametros *parametros = (Parametros *) params;

    const TickType_t xDelayTicks = pdMS_TO_TICKS(200);
    TickType_t xLastWakeTime = xTaskGetTickCount();

```



```

    int lectura_maxima=0;
    for(;;)
    {
        if(!xSemaphoreTake(parametros->mutex, portMAX_DELAY))
        {
            continue;
        }

        printf("Tarea 2 coge mutex: %lu\n", (unsigned
long)xTaskGetTickCount()*portTICK_PERIOD_MS);
        int *array_lecturas = (int*) pvPortMalloc(1000 * sizeof(int));
        if (!array_lecturas)
        {
            fprintf(stderr, "Tarea2: pvPortMalloc fallo\n");
            xSemaphoreGive(parametros->mutex);
            vTaskDelay(xDelayTicks);
            continue;
        }

        for (int i=0; i < 1000; ++i)
        {
            adc_oneshot_read(parametros->adc, ADC_CHANNEL_5,
array_lecturas+i);
        }

        if(xSemaphoreGive(parametros->mutex)==pdFALSE)
        {
            fprintf(stderr, "Tarea 2: error al soltar el mutex: %lu\n",
(unsigned long)xTaskGetTickCount()*portTICK_PERIOD_MS);
        }

        int lectura_mas_alta=array_lecturas[0];
        for(int i= 0; i<1000; ++i)
        {
            if(lectura_mas_alta < *(array_lecturas+i))
            {
                lectura_mas_alta= array_lecturas[i];
            }
        }

        lectura_maxima=lectura_mas_alta>lectura_maxima?
lectura_mas_alta:lectura_maxima;

        vPortFree(array_lecturas);
        printf("Tarea 2 lectura máxima: %d | tiempo:
%lu\n", lectura_maxima, (unsigned
long)xTaskGetTickCount()*portTICK_PERIOD_MS);

        xTaskDelayUntil(&xLastWakeTime, xDelayTicks);
    }
}

```

```

void tarea3(void* params)
{
    Parametros *parametros = (Parametros *) params;

    const    TickType_t xDelayTicks = pdMS_TO_TICKS(200);
            TickType_t xLastWakeTime = xTaskGetTickCount();

    int lectura_promedio=0;
    for(;;)
    {
        if(!xSemaphoreTake(parametros->mutex, portMAX_DELAY))
        {
            continue;
        }

        printf("Tarea 3 coge mutex: %lu\n", (unsigned
long)xTaskGetTickCount()*portTICK_PERIOD_MS);
        int *array_lecturas = (int*) pvPortMalloc(1000 * sizeof(int));
        if (!array_lecturas)
        {
            fprintf(stderr, "Tarea3: pvPortMalloc fallo\n");
            xSemaphoreGive(parametros->mutex);
            vTaskDelay(xDelayTicks);
            continue;
        }

        for (int i=0; i < 1000; ++i)
        {
            adc_oneshot_read(parametros->adc, ADC_CHANNEL_5,
array_lecturas+i);
        }

        if(xSemaphoreGive(parametros->mutex)==pdFALSE)
        {
            fprintf(stderr, "Tarea 3: error al soltar el mutex: %lu\n",
(unsigned long)xTaskGetTickCount()*portTICK_PERIOD_MS);
        }

        int lectura_total = 0;
        for(int i= 0; i<1000; ++i)
        {
            lectura_total += array_lecturas[i];
        }

        lectura_promedio=lectura_total/1000;

        vPortFree(array_lecturas);
        printf("Tarea 3 lectura promedio: %d | tiempo:
%lu\n", lectura_promedio, (unsigned
long)xTaskGetTickCount()*portTICK_PERIOD_MS);

        xTaskDelayUntil(&xLastWakeTime, xDelayTicks);
    }
}

```

```

    }
}

void app_main(void)
{
    Parametros parametros;
    adc_oneshot_unit_handle_t adc1_handle;

    adc_oneshot_unit_init_cfg_t init_config =
    {
        .unit_id = ADC_UNIT_1,
    };

    adc_oneshot_new_unit(&init_config, &adc1_handle);

    adc_oneshot_chan_cfg_t config =
    {
        .atten=ADC_ATTEN_DB_12,
        .bitwidth=ADC_BITWIDTH_DEFAULT,
    };

    adc_oneshot_config_channel(adc1_handle, ADC_CHANNEL_5, &config);

    TaskHandle_t xHandleT1 ,xHandleT2 ,xHandleT3 = NULL;

    parametros.mutex=xSemaphoreCreateMutex();
    if(parametros.mutex==NULL)
    {
        fprintf(stderr,"Error creando el mutex\n");
    }

    parametros.adc = adc1_handle;

    xTaskCreate(tarea1,"Tarea-1", STACK_SIZE ,(void * )&parametros,1,
    &xHandleT1);
    xTaskCreate(tarea2,"Tarea-2", STACK_SIZE ,(void * )&parametros,1,
    &xHandleT2);
    xTaskCreate(tarea3,"Tarea-3", STACK_SIZE ,(void * )&parametros,1,
    &xHandleT3);

    for(;;)
    {
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

```

EJERCICIO 5.1

```

/*
Copyright (C) 2026 Marcelo Fort Muñoz, Victor Arroyo Márquez

This program is free software: you can redistribute it and/or
modify it under the terms of the GNU General Public License as
published by the Free Software Foundation, either version 3 of
the License, or any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program. If not, see
<https://www.gnu.org/licenses/>.

*/

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "driver/gpio.h"
#include "esp_adc/adc_oneshot.h"

#define STACK_SIZE 4096

#define LED1 GPIO_NUM_4 // Parpadeo constante
#define LED2 GPIO_NUM_5 // Cambia con el botón
#define BOTON GPIO_NUM_0 // BOTÓN BOOT
#define ADC_CHANNEL ADC_CHANNEL_5 // GPIO6

typedef struct
{
    adc_oneshot_unit_handle_t adc;
    SemaphoreHandle_t sem_t3;
} ParamADC;

void tarea1(void *params)
{
    const TickType_t espera = pdMS_TO_TICKS(50); // 10Hz
    TickType_t lastWakeTime = xTaskGetTickCount();
    bool estado_led = false;

    for (;;)
    {
        gpio_set_level(LED1, estado_led);
        estado_led = !estado_led;
        vTaskDelayUntil(&lastWakeTime, espera);
    }
}

```

```

}
}

void tarea2(void *params)
{
    ParamADC *p = (ParamADC *) params;
    const TickType_t espera = pdMS_TO_TICKS(20);
    TickType_t lastWakeTime = xTaskGetTickCount();
    bool led2_state = false;

    int last_level = 1;

    for (;;)
    {
        int level = gpio_get_level(BOTON);

        if (last_level == 1 && level == 0)
        {
            printf("¡Botón pulsado!\n");
            led2_state = !led2_state;

            xSemaphoreGive(p->sem_t3);

            char buffer[512];
            vTaskGetRunTimeStats(buffer);
            fprintf(stdout, "%s\n", buffer);

            /*
             * Para ver el efecto de la suspensión, se puede simular
             una tarea pesada con un bucle vacío o una función de retardo.
             * Por ejemplo, podríamos agregar un retardo de 3 segundos
             aquí para simular que la tarea está ocupada haciendo algo:
             */
            for(long i=0; i<20000000; ++i)
            {
                __asm volatile ("nop");
            }

            /*

            vTaskDelay(pdMS_TO_TICKS(200)); // debounce
            lastWakeTime = xTaskGetTickCount(); //Actualizo el
momento de toma de tiempo

        }
        last_level=level;
        gpio_set_level(LED2, led2_state);
        vTaskDelayUntil(&lastWakeTime, espera);
    }
}

```

```

}

void tarea3(void *params)
{
    ParamADC *p = (ParamADC *) params;
    int valores[10];

    for (;;)
    {
        xSemaphoreTake(p->sem_t3, portMAX_DELAY);

        int suma = 0;
        for (int i = 0; i < 10; i++)
        {
            adc_oneshot_read(p->adc, ADC_CHANNEL, &valores[i]);
            suma += valores[i];
        }

        printf("Promedio ADC: %d\n", suma / 10);
    }
}

void app_main()
{
    gpio_reset_pin(LED1);
    gpio_reset_pin(LED2);
    gpio_set_direction(LED1, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED2, GPIO_MODE_OUTPUT);

    gpio_reset_pin(BOTON);
    gpio_set_direction(BOTON, GPIO_MODE_INPUT);
    gpio_pullup_en(BOTON);

    adc_oneshot_unit_handle_t adc_handle;
    adc_oneshot_unit_init_cfg_t init_config =
    {
        .unit_id = ADC_UNIT_1,
    };
    adc_oneshot_new_unit(&init_config, &adc_handle);

    adc_oneshot_chan_cfg_t chan_cfg =
    {
        .atten = ADC_ATTEN_DB_12,
        .bitwidth = ADC_BITWIDTH_DEFAULT,
    };
    adc_oneshot_config_channel(adc_handle, ADC_CHANNEL, &chan_cfg);

    SemaphoreHandle_t sem_t3 = xSemaphoreCreateBinary();
}

```

```
xSemaphoreGive(sem_t3); // Up al iniciar para que no se quede
bloqueada esperando el primer botón

ParamADC param_adc =
{
    .adc = adc_handle,
    .sem_t3 = sem_t3
};

xTaskCreate(tarea1, "Tarea 1", STACK_SIZE, NULL, 1, NULL);
xTaskCreate(tarea2, "Tarea 2", STACK_SIZE, (void *)&param_adc, 1,
NULL);
xTaskCreate(tarea3, "Tarea 3", STACK_SIZE, (void *)&param_adc, 1,
NULL);

for(;;) // Para que no mate la tarea
{
    vTaskDelay(pdMS_TO_TICKS(10));
}
}
```

SESIÓN 3

EJERCICIO 2.1

```
/*
Código base proporcionado por el profesorado de la asignatura.
Autor original: Javier Burgoa.
Uso exclusivamente académico para la práctica 3.

Creado por Javier Burgoa el 24/11/2024
Código base para la práctica 3.
5 tareas se ejecutan de forma concurrente.
5 tareas de igual prioridad
5 tareas con la misma carga computacional

Al pulsar BOOT se muestran las estadísticas por pantalla

El CMakeLists debe incluir >> PRIV_REQUIRES driver esp_adc esp_timer
*/

#include <stdio.h>
#include <string.h>
#include <math.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/gpio.h"
#include "esp_adc/adc_oneshot.h"
#include "driver/ledc.h"
#include "esp_timer.h"
// #include "driver/uart.h"
```

```
#include "esp_clk_tree.h"
#include "esp_log.h"

//PERIODOS TAREAS en ms
// CAMBIO: 400 -> 200.
#define TASK1_T 200
#define TASK2_T 200
#define TASK3_T 200
#define TASK4_T 200
#define TASK5_T 200

//CONTADORES
// CAMBIO: 15000 -> 100 000.
#define ITERATE_1 100000
#define ITERATE_2 100000
#define ITERATE_3 100000
#define ITERATE_4 100000
#define ITERATE_5 100000

//GPIO SALIDAS Y ENTRADAS DIGITALES
#define PULSADOR 0

#define STACK_SIZE      4*1024           //N x 1kByte es el tamaño de
la pila

bool flag_stats = false;
void imprime_estadisticas(void);

//-----Task1-----
//-----
void vTaskCode1( void * pvParameters )
{
    TickType_t          xLastWakeTime;
    const TickType_t    xDelayTicks = TASK1_T/portTICK_PERIOD_MS;

    xLastWakeTime = xTaskGetTickCount ();           // Initialise the
xLastWakeTime variable with the current time.
    while(1)
    {
        if (!flag_stats)
            printf("\tI\t\t\t\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //Consume CPU cycles
        for (long i = 0; i < ITERATE_1; i++) {
            __asm__ __volatile__ ("NOP");
        }

        if (!flag_stats)
            printf("\t0\t\t\t\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //bloquea y espera hasta TASK1_T ms
        xTaskDelayUntil( &xLastWakeTime, xDelayTicks );
    }
}
```



```

}
//-----Task2-----
//-----
void vTaskCode2( void * pvParameters )    //
{
    TickType_t      xLastWakeTime;
    const TickType_t xDelayTicks = TASK2_T/portTICK_PERIOD_MS;

    xLastWakeTime = xTaskGetTickCount ();    // Initialise the
xLastWakeTime variable with the current time.
    while(1)
    {
        if (!flag_stats)
            printf("\t\tI\t\t\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //Consume CPU cycles
        for (long i = 0; i < ITERATE_2; i++) {
            __asm__ __volatile__ ("NOP");
        }
        if (!flag_stats)
            printf("\t\t0\t\t\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //bloquea y espera hasta TASK2_T ms
        xTaskDelayUntil( &xLastWakeTime, xDelayTicks );
    }
}
//-----Task3:-----
//-----
void vTaskCode3( void * pvParameters )
{
    TickType_t      xLastWakeTime;
    const TickType_t xDelayTicks = TASK3_T/portTICK_PERIOD_MS;

    xLastWakeTime = xTaskGetTickCount ();    // Initialise the
xLastWakeTime variable with the current time.

    while(1)
    {
        if (!flag_stats)
            printf("\t\t\tI\t\t\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //Consume CPU cycles
        for (long i = 0; i < ITERATE_3; i++) {
            __asm__ __volatile__ ("NOP");
        }
        if (!flag_stats)
            printf("\t\t\t0\t\t\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //bloquea y espera hasta TASK3_T ms
        xTaskDelayUntil( &xLastWakeTime, xDelayTicks );
    }
}

```

```
//-----Task4:-----
//-----
//-----
void vTaskCode4( void * pvParameters )
{
    TickType_t      xLastWakeTime;
    const TickType_t xDelayTicks = TASK4_T/portTICK_PERIOD_MS;

    xLastWakeTime = xTaskGetTickCount ();           // Initialise the
xLastWakeTime variable with the current time.
    while(1)
    {
        if (!flag_stats)
            printf("\t\t\t\t\tI\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //Consume CPU cycles
        for (long i = 0; i < ITERATE_4; i++) {
            __asm__ __volatile__ ("NOP");
        }
        if (!flag_stats)
            printf("\t\t\t\t\t0\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //bloquea y espera hasta TASK4_T ms
        xTaskDelayUntil( &xLastWakeTime, xDelayTicks );
    }
}

//-----Task5:-----
//-----
//-----
void vTaskCode5( void * pvParameters )
{
    TickType_t      xLastWakeTime;
    const TickType_t xDelayTicks = TASK4_T/portTICK_PERIOD_MS;
    xLastWakeTime = xTaskGetTickCount ();           // Initialise the
xLastWakeTime variable with the current time.
    while(1)
    {
        if (!flag_stats)
            printf("\t\t\t\t\tI\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //Consume CPU cycles
        for (long i = 0; i < ITERATE_5; i++) {
            __asm__ __volatile__ ("NOP");
        }

        if (!flag_stats)
            printf("\t\t\t\t\t0\t\t\t%llu\n", esp_timer_get_time()/1000 );

        //bloquea y espera hasta TASK5_T ms
        xTaskDelayUntil( &xLastWakeTime, xDelayTicks );
    }
}
```

```
//-----Main loop-----
//-----

#define GPIO_OUTPUT_IO_0    4      //GPIO 4 - salida 1 - 3Hz
#define GPIO_OUTPUT_IO_1    5      //GPIO 5 - salida 2 - 6Hz
#define PULSADOR            0      //GPIO 0 - entrada digital

void app_main(void)
{
    /***** Config GPIO *****/
    gpio_set_direction(GPIO_OUTPUT_IO_0,GPIO_MODE_OUTPUT) ;
//GPIO_OUTPUT_IO_0
    gpio_set_direction(GPIO_OUTPUT_IO_1,GPIO_MODE_OUTPUT) ;
//GPIO_OUTPUT_IO_1

    gpio_config_t io_conf = {
        .pin_bit_mask = 1ULL << PULSADOR,
        .mode          = GPIO_MODE_INPUT,
        .pull_up_en     = true,           //pull-up habilitada
        .pull_down_en   = false,         //pull-down deshabilitada
    };
    gpio_config(&io_conf);

    //Crea Handlers a las tareas
    TaskHandle_t xHandle1 = NULL; //Handler a la tarea 1
    TaskHandle_t xHandle2 = NULL; //Handler a la tarea 2
    TaskHandle_t xHandle3 = NULL; //Handler a la tarea 3
    TaskHandle_t xHandle4 = NULL; //Handler a la tarea 4
    TaskHandle_t xHandle5 = NULL; //Handler a la tarea 5

    //Crea tareas
    xTaskCreate( vTaskCode1, "TASK1", STACK_SIZE, NULL, 1,
&xHandle1 ); //Prioridad 1
    xTaskCreate( vTaskCode2, "TASK2", STACK_SIZE, NULL, 1,
&xHandle2 ); //Prioridad 1
    xTaskCreate( vTaskCode3, "TASK3", STACK_SIZE, NULL, 1,
&xHandle3 ); //Prioridad 1
    xTaskCreate( vTaskCode4, "TASK4", STACK_SIZE, NULL, 1,
&xHandle4 ); //Prioridad 1
    xTaskCreate( vTaskCode5, "TASK5", STACK_SIZE, NULL, 1,
&xHandle5 ); //Prioridad 1

    int entrada_digital_t0 = 0;
    int entrada_digital_t1 = 0;

    while (1)
    {
        //main loop

        entrada_digital_t0 = gpio_get_level(PULSADOR);

        //en flanco de bajada
    }
}
```

```

        if ((entrada_digital_t0==0)&&(entrada_digital_t1==1))
        {
            imprime_estadisticas();
        }
        entrada_digital_t1 = entrada_digital_t0;

        vTaskDelay(100/portTICK_PERIOD_MS);    //Wait 1 sec
    }
}

void imprime_estadisticas(void)
{
    char Buff[512]      = {0};
    char task_list[512] = {0};
    uint32_t cpu_freq_hz = 0;

    //----- Statistics -----

    //Consulta el stado de todas las tareas
    vTaskList(task_list);
    vTaskGetRunTimeStats(Buff);
    esp_clk_tree_src_get_freq_hz(SOC_CPU_CLK_SRC_PLL,
ESP_CLK_TREE_SRC_FREQ_PRECISION_CACHED, &cpu_freq_hz);

    //imprime por el terminal.
    printf("*****\n");
    printf("Estado de las tareas:\n%s\n", task_list);

    //Imprime la carga de la CPU.
    printf("%s ", Buff);

    //CPU Freq
    printf("La CPU se ha configurado a : %lu MHz\n", cpu_freq_hz /
1000000);

    //ticks rate
    printf("TIME SLICE: %d Hz, %.1f ms\n", configTICK_RATE_HZ, (float)
(1000.0/configTICK_RATE_HZ));

    printf("*****\n");
}

```

B. Anexo II: Licencias del contenido

El código fuente del proyecto está licenciado bajo la GNU General Public License, versión 3 o posterior, salvo aquellos fragmentos proporcionados por el profesorado, que mantienen su autoría original y no se consideran parte del código desarrollado por el estudiante.

Los fragmentos de código de terceros incluidos en el proyecto están debidamente identificados en el código fuente.

Los fragmentos de código incluidos en este documento se muestran únicamente con fines ilustrativos.

Este documento está licenciado bajo la GNU Free Documentation License, versión 1.3 o posterior.

El texto completo de ambas licencias está disponible en el repositorio del proyecto:
<https://github.com/marcelo9987/Practicas_STFTR>

